

# Uncertainty-Aware Path Planning for Stewart Platforms

Wm. Sam Schneidereith  
*University of Maryland, College Park*

Stewart platforms are widely used in precision aerospace applications due to their high stiffness and six-degree-of-freedom actuation capabilities. In tasks such as in-orbit assembly and robotic manipulation, maintaining accurate knowledge of the platform state throughout motion is critical. Traditional sampling-based motion planners typically optimize geometric objectives such as path length, which does not directly account for the uncertainty that accumulates during actuation. This work presents an uncertainty-aware motion planning framework for Stewart platforms that incorporates belief propagation using an Extended Kalman Filter (EKF). The EKF is used to propagate state covariance along candidate trajectories, enabling the trace of the error covariance matrix to be used as a cost function during planning. Sampling-based motion planners from the Rapidly-Exploring Random Tree (RRT) family are evaluated within the full six-dimensional platform configuration space. Comparative experiments show that RRT# demonstrates superior convergence behavior relative to RRT and RRT\*. When uncertainty is used as the optimization metric, the planner successfully identifies trajectories with progressively lower estimation uncertainty over time. Experimental results further indicate that path length and path uncertainty are not strongly correlated, suggesting that minimizing estimation uncertainty may require trajectories that differ from geometrically optimal paths. These results demonstrate the potential of uncertainty-aware sampling-based planning for improving reliability in high-precision robotic systems.

## I. Nomenclature

|                |   |                                    |
|----------------|---|------------------------------------|
| $\mathcal{G}$  | = | Graph Structure                    |
| $\mathcal{V}$  | = | Set of Vertices                    |
| $\mathcal{E}$  | = | Set of Edges                       |
| $Q$            | = | Priority Queue                     |
| $x_{start}$    | = | Start Node                         |
| $x_{goal}$     | = | Goal Node                          |
| $x_{rand}$     | = | Randomly Sampled Node in Workspace |
| $\Delta$       | = | Step Length                        |
| $\gamma_{RRT}$ | = | Shrinking Ball Radius Constant     |
| $c$            | = | Cost Function                      |
| $h$            | = | Heuristic Condition                |
| $g$            | = | Node Global Cost                   |
| $lmc$          | = | Node Local Minimum Cost            |
| $x$            | = | Platform State Vector              |
| $y$            | = | Innovation                         |
| $z$            | = | Measurement                        |
| $P$            | = | State Uncertainty Covariance       |
| $Q$            | = | Process Noise Covariance           |
| $R$            | = | Measurement Noise Covariance       |
| $J$            | = | Stewart Platform Jacobian          |
| $F$            | = | Jacobian of Motion Model           |
| $H$            | = | Jacobian of Observation Model      |
| $K$            | = | Kalman Gain                        |
| $V$            | = | Platform Twist                     |
| $l_i$          | = | Actuator Length                    |

$u_i$  = Actuator Unit Vector  
 $t$  = Platform Translation  
 $R_{zyx}$  = Platform Rotation Matrix

## II. Introduction

Stewart platforms are widely used in aerospace engineering due to their precise six-degree-of-freedom actuation and high load capacity. They are a class of parallel robotic manipulators consisting of two rigid platforms connected by six independently actuated linear legs. This architecture enables accurate control of both position and orientation, making Stewart platforms well suited for applications requiring high precision and stiffness. Examples include six-degree-of-freedom flight simulators and soft-capture docking mechanisms used by NASA.

This investigation is motivated by research conducted at NASA Langley Research Center exploring the use of compact Stewart platforms as end effectors on robotic manipulators for space applications. Such systems are being considered for tasks including in-orbit assembly and manipulation. Previous work on this system was conducted by Ryan Erandis, a former aerospace engineering master's student at the University of Maryland [1]. The present work evaluates and expands upon that framework, with a particular focus on motion planning and uncertainty propagation.

In these applications, precision is critical. During orbital assembly or manipulation, positional errors can result in misalignment or damage to high-value hardware. For this reason, minimizing state uncertainty during motion is often more important than minimizing travel distance or execution time. All robotic motion contains inherent uncertainty arising from actuator errors, process disturbances, and sensor measurement noise. Effective motion planning must therefore consider how these sources of uncertainty accumulate throughout a commanded trajectory.

### A. Kinematics

Parallel robotic manipulators such as Stewart platforms present additional challenges in control and state estimation due to their kinematic structure. In robotics, forward kinematics refer to determining the pose of the end effector from known actuator states. In serial manipulators, such as conventional robotic arms, this relationship is typically straightforward: the pose of the end effector can be computed directly from joint angles and link lengths.

Inverse kinematics, by contrast, determine the actuator configuration required to achieve a desired end-effector pose. In serial manipulators this problem is often non-unique, meaning that multiple joint configurations may correspond to the same end-effector pose. An easy visualization is to position the hand in front of oneself moving the wrist, elbow and shoulder joints to achieve various poses with the same hand state.

Parallel manipulators exhibit the opposite behavior. For Stewart platforms, the inverse kinematics problem is relatively simple. Given a desired platform pose, the required actuator lengths can be computed in closed form. However, the forward kinematics problem is significantly more complex. For a given set of actuator lengths, the system may correspond to multiple possible platform poses. According to Merlet[2], there are up to 12 possible assembly modes for the same set of actuator lengths.

This ambiguity introduces a significant challenge in state estimation. If the controller assumes that the platform is in one configuration while it actually occupies another valid configuration corresponding to the same actuator lengths, initiating further motion could damage the mechanism or surrounding equipment. Reliable operation therefore requires maintaining accurate knowledge of the platform state throughout motion and limiting the accumulation of estimation uncertainty.

### B. Sampling Based Motion Path Planning

Sampling-based motion planning provides a framework for computing feasible trajectories in complex, sometimes high-dimensional spaces. These methods assume that only limited information about the workspace is available, typically including the start configuration, goal configuration, and collision-checking that determines whether a state lies within the valid workspace.

Many sampling-based planners operate by incrementally constructing a tree that expands through the configuration space via random sampling. In this work, algorithms from the Rapidly-Exploring Random Tree (RRT) family are used, specifically RRT, RRT\*, and RRT# (pronounced "RRT star" and "RRT sharp," respectively). Each successive algorithm improves upon the previous by incorporating mechanisms that improve convergence toward the optimal path. In this investigation, these algorithms are first evaluated using a traditional distance-based cost to compare their convergence behavior and solution quality.

Dimensionality plays an important role in motion planning performance. While many demonstrations of sampling-based planners operate in two- or three-dimensional spaces, the Stewart platform requires planning over the full six-dimensional pose space consisting of three position and three orientation components. Increasing dimensionality significantly increases the complexity of the planning problem, as the volume of the configuration space grows exponentially. Although planning time depends on available computational resources, maintaining consistent hardware across experiments ensures that algorithmic comparisons remain valid.

### C. Path Uncertainty as a Cost Function

Traditional sampling-based motion planners typically optimize geometric objectives, such as path length. Though this approach is appropriate for many robotic systems, it does not directly address the uncertainty that accumulates during motion. For high-precision applications, minimizing the accumulated path uncertainty can be more important than minimizing the distance traveled. Additionally, it is not sufficient to consider uncertainty only at the final configuration. Large uncertainties anywhere along the trajectory can lead to inaccurate state estimation, therefore uncertainty must be minimized throughout the entire trajectory.

In this work, trajectory quality is therefore evaluated based on the uncertainty of the platform state along the planned path. Uncertainty is propagated using an Extended Kalman Filter (EKF) that models process disturbances and measurement noise during motion. The resulting state covariance provides a quantitative measure of estimation uncertainty, allowing candidate trajectories to be compared based on their accumulated uncertainty. By incorporating uncertainty directly into the path evaluation process, motion planning can prioritize precision and state confidence rather than purely geometric efficiency.

### D. Contributions

- Modularized the simulation environment for clear compartmentalization of properties and methods with respect to the planner and Stewart Platform.
- Performed comparative evaluation of RRT, RRT\*, and RRT# for six-degree-of-freedom platform motion planning.
- Re-derived Stewart Platform inverse kinematics and added visualization plots for system state insight.
- Implemented standalone tests of Extended Kalman Filter to verify functionality.
- Demonstrated consistent reduction in path uncertainty during path planning.
- Experimentally analyzed the relationship between path uncertainty and path length.

## III. Problem Formulation

The objective of this work is to generate motion trajectories for a Stewart platform that minimize estimation uncertainty during motion. The configuration of the platform is represented by the state vector

$$x = \begin{bmatrix} x & y & z & \phi & \theta & \psi \end{bmatrix}^T \quad (1)$$

where the first three components represent platform position and the remaining components represent orientation using Euler angles. Given an initial configuration  $x_{start}$  and a desired goal configuration  $x_{goal}$ , the motion planning problem is to compute a feasible trajectory through the platform configuration space while satisfying workspace bounds and actuator length constraints.

Rather than optimizing purely geometric metrics such as path length, this work evaluates trajectories based on the uncertainty of the platform state propagated through an Extended Kalman Filter (EKF). The cost associated with a node  $x$  in the planning tree is defined as

$$c(x) = g(parent) + \text{trace}(P_x) \quad (2)$$

where  $P_x$  is the state covariance matrix associated with node  $x$ . Minimizing this cost encourages trajectories that maintain low estimation uncertainty throughout execution. The motion planning problem can therefore be expressed as finding a sequence of configurations connecting  $x_{start}$  to  $x_{goal}$  that minimizes accumulated uncertainty while satisfying system constraints.

## IV. Sampling-Based Motion Planning

Sampling-based motion planning algorithms construct feasible paths through a configuration space by repeatedly sampling new states and connecting them to an existing graph structure. This section introduces three related algorithms that form the foundation of the planning framework used in this work: Rapidly-Exploring Random Trees (RRT), RRT\*, and RRT#.

Motion planning for the Stewart platform is performed in the full six-dimensional configuration space consisting of platform position and orientation. Each configuration is represented by the state vector

$$x = \begin{bmatrix} x & y & z & \phi & \theta & \psi \end{bmatrix}^T$$

where the first three components represent the platform position and the remaining three represent orientation using Euler angles.

The planner constructs a tree of reachable states beginning from the start configuration and incrementally expands this tree toward randomly sampled configurations within the platform workspace. Each node in the tree stores the platform state, its associated uncertainty covariance matrix, and the cumulative cost required to reach that node.

To efficiently perform nearest-neighbor queries during tree expansion, the planner maintains a k-d tree structure over the state space. This enables rapid identification of nearby nodes during both expansion and rewiring operations.

The planning process terminates either after a fixed time limit or once a predefined number of nodes has been added to the tree. Throughout the planning process, the best solution path is continuously updated whenever a lower-cost trajectory to the goal configuration is discovered.

### A. Rapidly-Exploring Random Trees (RRT)

The Rapidly-Exploring Random Trees algorithm [3] is one of the first sampling-based motion planning algorithms. It is simple and effective at rapidly exploring a workspace, but it does not optimize much beyond that.

Starting from an initial node, the algorithm repeatedly samples a random point in the workspace. The nearest node in the existing tree is identified, and the algorithm attempts to steer from this nearest node toward the sampled point with a maximum step length of  $\Delta$ . If the resulting state is valid, a new node is added to the tree and connected to its parent. When path length is used as the cost metric, the cost of the new node is simply the cumulative path length from the start node to the new node.

---

#### Algorithm 1 Rapidly-Exploring Random Trees (RRT)

---

```

1: function RRT( $\mathcal{G}, x_{start}$ )
2:    $\mathcal{V} \leftarrow x_{start}; \mathcal{E} \leftarrow \emptyset$ 
3:   for  $k = 1 \dots K$  do
4:      $x_{rand} \leftarrow \text{RAND}(\mathcal{G})$ 
5:      $x_{near} \leftarrow \text{NEARESTNEIGHBOR}(\mathcal{V}, x_{rand})$ 
6:      $x_{new} \leftarrow \text{STEER}(x_{near}, x_{rand}, \Delta)$ 
7:      $\mathcal{V} \leftarrow \mathcal{V} \cup \{x_{new}\}$ 
8:      $\mathcal{E} \leftarrow \mathcal{E} \cup \{(x_{near}, x_{new})\}$ 
9:   end for
10:  return  $\mathcal{G} \leftarrow (\mathcal{V}, \mathcal{E})$ 
11: end function

```

---

### B. RRT\*

RRT\*[4] improves upon the basic RRT algorithm by introducing a local optimization step when new nodes are added to the tree. After generating a candidate node  $x_{new}$ , the algorithm searches for nearby nodes within a ball of radius  $r$  and evaluates them as potential parent candidates.

Rather than always connecting to the nearest node, RRT\* selects the parent that results in the lowest total cost-to-come. After the parent is selected, nearby nodes are also evaluated to determine whether they would achieve a lower cost by rewiring through  $x_{new}$ . This local rewiring step gradually improves the tree structure over time.

Due to this mechanism, RRT\* is asymptotically optimal and converges toward the lowest-cost solution as the number of samples approaches infinity.

Algorithm 2 outlines the RRT\* procedure.

---

**Algorithm 2** RRT\*

---

```

1: function RRTSTAR( $\mathcal{G}, x_{start}$ )
2:    $\mathcal{V} \leftarrow x_{start}; \mathcal{E} \leftarrow \emptyset$ 
3:   for  $k = 1 \dots K$  do
4:      $x_{rand} \leftarrow \text{RAND}(\mathcal{G})$ 
5:      $x_{near} \leftarrow \text{NEARESTNEIGHBOR}(\mathcal{V}, x_{rand})$ 
6:      $x_{new} \leftarrow \text{STEER}(x_{near}, x_{rand}, \Delta)$ 
7:      $\{\text{neighbors}\} \leftarrow \text{NEIGHBORS}(\mathcal{V}, x_{new}, \text{BALLRAD}(\Delta, \gamma_{RRT}))$ 
8:     for  $\text{neighbor} \in \text{neighbors}$  do
9:       if  $\text{neighbor}.g + c(\text{neighbor}, x_{new}) < x_{new}.g$  then
10:         $x_{new}.g \leftarrow \text{neighbor}.g + c(\text{neighbor}, x_{new})$ 
11:         $x_{new}.parent \leftarrow \text{neighbor}$ 
12:       end if
13:        $\mathcal{E} \leftarrow \mathcal{E} \cup \{(x_{new}, \text{neighbor})\}$ 
14:     end for
15:      $\mathcal{V} \leftarrow \mathcal{V} \cup \{x_{new}\}$ 
16:   end for
17:   return  $\mathcal{G} \leftarrow (\mathcal{V}, \mathcal{E})$ 
18: end function

```

---

### C. RRT#

RRT#[5] is the most advanced of the three algorithms considered in this work. It extends RRT\* by introducing a dynamic replanning procedure that propagates cost improvements through the graph.

Before discussing the replanning process, two key concepts must be defined: predecessors and successors. A predecessor is any existing node within the connection radius of a newly added node  $x_{new}$ . A successor is any node added after the current node that lies within the same radius. In other words, when  $x_{new}$  is inserted into the graph, all nearby nodes are added to its list of successors, and  $x_{new}$  is added to their lists of predecessors.

For undirected graphs, where edge costs are identical regardless of traversal direction, predecessors and successors are equivalent. In this case, the term successors is used as a general descriptor for neighboring nodes.

The replanning procedure begins when a newly inserted node provides a lower-cost path to one of its successor nodes. When this occurs, the successor updates its parent to the lower-cost node. After the parent update, all of the node's children must update their costs to reflect the change. During this process, their successors are examined to determine whether they can also achieve a lower cost by rewiring through the updated node. This creates a chain of cost updates that propagates through the graph.

The propagation process continues only while a node's potential new cost is lower than its current cost-to-goal estimate. Nodes requiring cost updates are inserted into a priority queue, which is maintained by the UpdateQueue procedure described in Algorithm 6.

The RRT# framework is composed of several interacting procedures, summarized in the following algorithms:

### D. Performance Analysis

One of the goals of this investigation was to run a more comprehensive set of repeated trials than appeared in Ernandis' dissertation. Averaging the experimental data over many trials removes some of the randomness of the *random tree* algorithms, and allows the algorithms' performances to be compared holistically. The three algorithms were evaluated by comparison of convergence speed and path optimality across many trials. All planning was done in the 6D weighted position-orientation configuration space.

Figure 1 shows the averaged results of 50 trials for each algorithm. The normalized path cost divides path length by the straight line euclidean distance between the start and goal node states, giving different start/goal configurations equal weights. For Figure 2, a successful trial was categorized as one that comes within 10% of the straight line distance (when normalized).

---

**Algorithm 3** RRT#

---

```
1: function RRTSHARP( $x_{start}, x_{goal}$ )
2:    $\mathcal{V} \leftarrow x_{start}; \mathcal{E} \leftarrow \emptyset$ 
3:    $\mathcal{G} \leftarrow \{\mathcal{V}, \mathcal{E}\}$ 
4:   for  $k = 1 \dots K$  do
5:      $x_{rand} \leftarrow \text{RAND}(\mathcal{G})$ 
6:      $x_{new} \leftarrow \text{EXTEND}(\mathcal{G}, x_{rand}, \Delta)$ 
7:     REPLAN( $params$ )
8:   end for
9:   return  $\mathcal{G}$ 
10: end function
```

---

---

**Algorithm 4** Extend

---

```
1: function EXTEND( $\mathcal{G}, x_{rand}, \Delta$ )
2:    $(\mathcal{V}, \mathcal{E}) \leftarrow \mathcal{G}$ 
3:    $x_{near} \leftarrow \text{NEARESTNEIGHBOR}(\mathcal{V}, x_{rand})$ 
4:    $x_{new} \leftarrow \text{STEER}(x_{near}, x_{rand}, \Delta)$ 
5:    $x_{new}.lmc \leftarrow x_{near}.g + c(x_{near}, x_{new})$ 
6:    $\{neighbors\} \leftarrow \text{NEIGHBORS}(\mathcal{V}, x_{new}, \text{BALLRAD}(\Delta, \gamma_{RRT}))$ 
7:   for  $n \in neighbors$  do
8:     if  $n.g + c(n, x_{new}) < x_{new}.lmc$  then
9:        $x_{new}.lmc \leftarrow n.g + c(n, x_{new})$ 
10:       $x_{new}.parent \leftarrow n$ 
11:    end if
12:     $x_{new}.successors \leftarrow x_{new}.successors \cup \{n\}$ 
13:     $n.successors \leftarrow n.successors \cup \{x_{new}\}$ 
14:     $\mathcal{E} \leftarrow \mathcal{E} \cup \{(x_{new}, n)\}$ 
15:  end for
16:  UPDATEQUEUE( $Q, x_{new}$ )
17:   $\mathcal{V} \leftarrow \mathcal{V} \cup \{x_{new}\}$ 
18:  return  $\mathcal{G} \leftarrow (\mathcal{V}, \mathcal{E})$ 
19: end function
```

---

---

**Algorithm 5** Replan

---

```
1: function REPLAN( $\mathcal{G}, Q$ )
2:   while PEEK( $Q$ ).key < MIN( $x_{goal}.g, x_{goal}.lmc$ ) do
3:      $x \leftarrow \text{POLL}(Q)$ 
4:      $x.g \leftarrow x.lmc$ 
5:     for  $s \in x.successors$  do
6:       if  $x.g + c(x, s) < s.lmc$  then
7:          $s.lmc \leftarrow x.g + c(x, s)$ 
8:          $s.parent \leftarrow x$ 
9:         UPDATEQUEUE( $Q, s$ )
10:      end if
11:    end for
12:  end while
13: end function
```

---

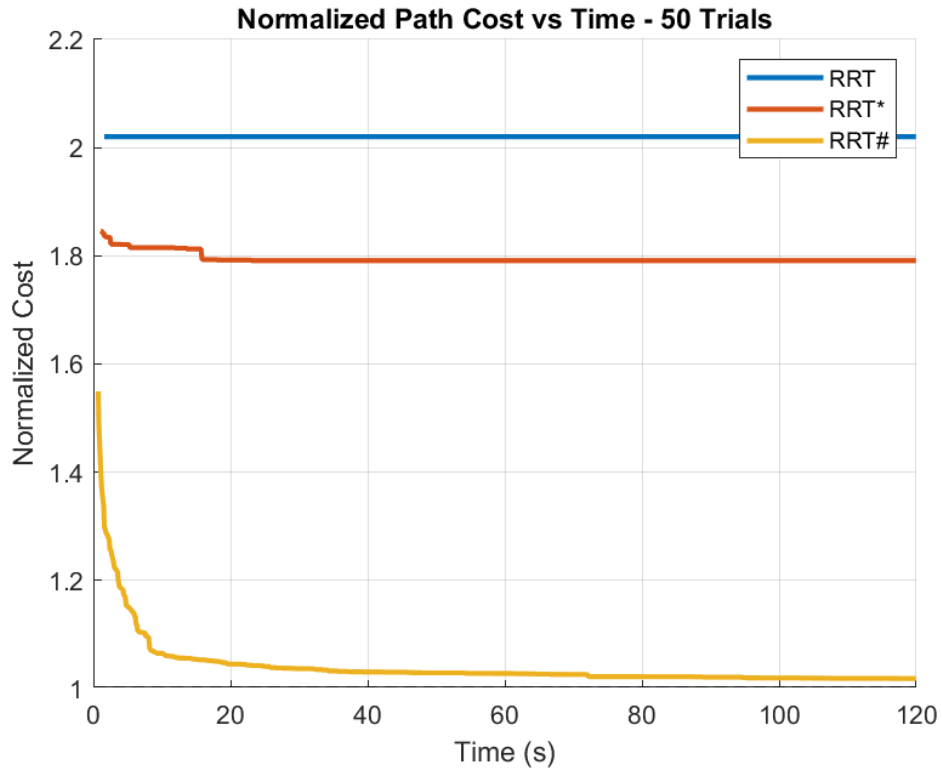
---

**Algorithm 6** UpdateQueue

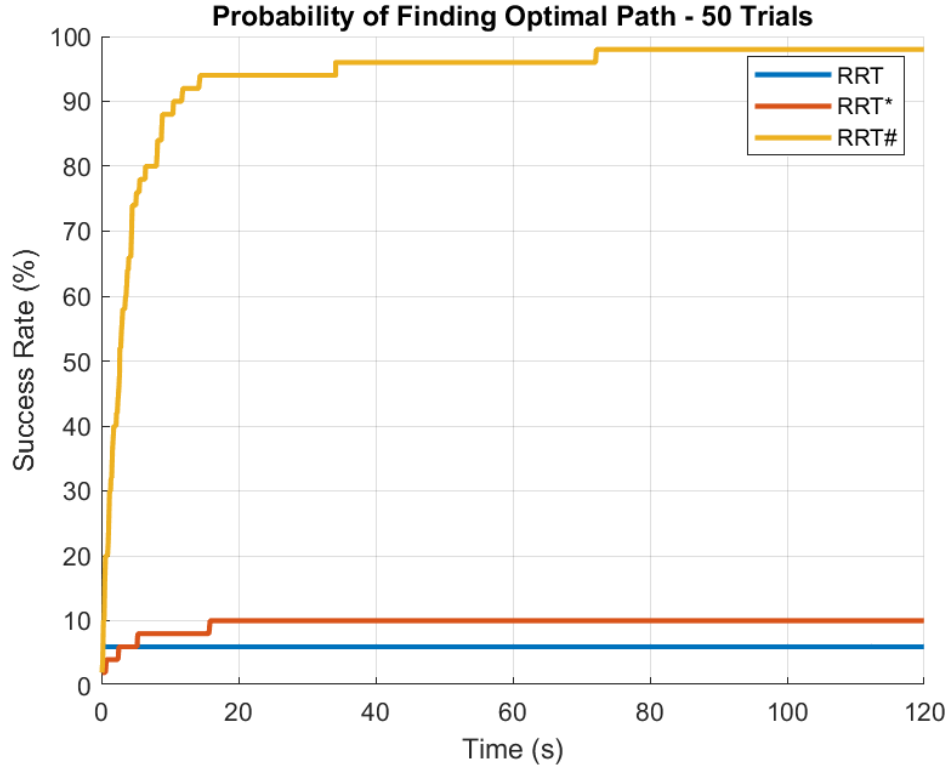
---

```
function UPDATEQUEUE( $Q, x$ )  
   $key = \text{MIN}(x.g, x.lmc) + H(x)$   
  if  $x.g \neq x.lmc$  &  $x \in Q$  then  
     $Q.update(key)$   
  else if  $x.g \neq x.lmc$  &  $x \notin Q$  then  
     $Q.add(x, key)$   
  else if  $x.g = x.lmc$  &  $x \in Q$  then  
     $Q.remove(x)$   
  end if  
end function
```

---



**Fig. 1** Normalized path cost as a function of computation time averaged over 50 trials for each algorithm.



**Fig. 2** Percentage of trials in which each algorithm converges to the shortest path.

Figure 1 demonstrates the significant performance differences between the three algorithms. RRT rapidly finds an initial feasible path but struggles to improve the solution as additional samples are added. RRT\* typically produces lower-cost paths earlier but similarly experiences diminishing improvement over time.

Only RRT# consistently converges toward the shortest path length. In fact, after 60 seconds of running, 100% of the RRT# trials met the success criteria. Due to its asymptotic optimality and its ability to propagate cost improvements efficiently throughout the tree, RRT# is selected as the primary planning algorithm used in the remainder of this investigation.

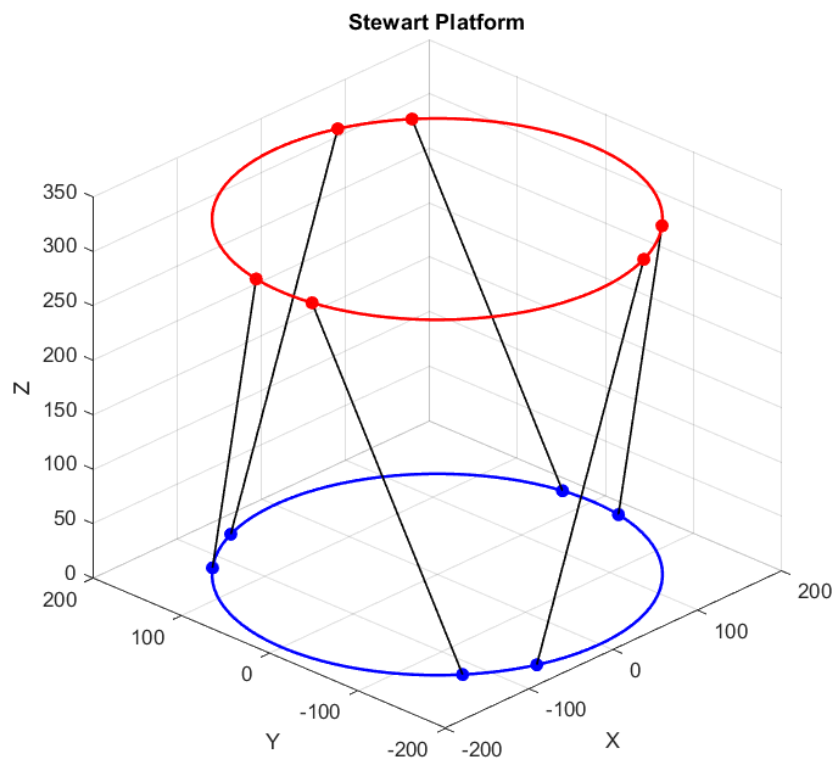
## V. Stewart Platform Modeling

### A. Platform Geometry

Stewart Platforms are composed of two parallel rigid platforms connected by six non-parallel linear actuators. On both the base and moving platform the actuators attach in three symmetric pairs. The ordering of these pairs differs between the base and top plates, producing the characteristic crossed leg configuration required for full six-degree-of-freedom motion. Figure 3 shows the geometry used in this investigation.

The linear actuators are connected to both the base and moving platform through universal joints. These joints allow free rotation of the actuator ends and ensure that only axial forces are transmitted through the legs, preventing torque from being applied directly to the platforms. This simplifies the kinematic model and allows each actuator to be treated as a pure length constraint between its attachment points.

For the specific Stewart platform analyzed in this work, the spacing between each pair of joints on both platforms is 63.5 mm. The radius of both the base and moving platforms is 185 mm, corresponding to the radial distance of each joint from the platform center. These geometric parameters fully define the fixed joint locations used in the kinematic calculations.



**Fig. 3 Geometry of Stewart Platform Investigated**

## B. Kinematics

### 1. Inverse Kinematics

The inverse kinematics of a Stewart Platform determine the actuator lengths required to achieve a desired platform pose. For this architecture the inverse kinematics are closed form and relatively straightforward. The platform pose is defined by the position vector  $t = [x \ y \ z]^T$  and the orientation represented using roll, pitch, and yaw angles  $(\phi, \theta, \psi)$ .

The first step is to determine the locations of the platform joints expressed in the base coordinate frame. Joint coordinates are initially defined in their respective platform frames. A rotation and translation transformation is then applied to map the platform joint coordinates into the base frame. The rotation matrix used in this work follows a Z-Y-X Euler angle convention and is given in Equation 3.

$$R_{zyx}(\phi, \theta, \psi) = R_z(\psi) R_y(\theta) R_x(\phi)$$

$$\text{where: } R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}, \quad R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}, \quad R_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3)$$

Using this rotation matrix, the coordinates of each platform joint  $P_i$  can be expressed in the base frame according to the transformation shown in Equation 4.

$${}^B P_i = R_{zyx} P_i + t$$

$$\text{where: } t = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \quad (4)$$

Once the platform joint positions are known in the base frame, the actuator vectors can be computed by subtracting the corresponding base joint locations  $B_i$ . The actuator length is simply the magnitude of this vector. Repeating this process for all six legs yields the full set of actuator lengths required to achieve the desired pose.

### 2. The Jacobian

The Stewart Platform Jacobian shown in Equation 6, describes how small changes in platform pose affect the lengths of the six actuators. In other words, it quantifies the sensitivity of each actuator length to changes in translational and rotational motion of the platform. This relationship becomes important when determining actuator velocities and when analyzing how uncertainty in actuator motion propagates to the platform state.

To construct the Jacobian, the vector from the base joint to the platform joint for each leg is first computed. The direction of each actuator is then obtained by normalizing this vector. Using these quantities, each row of the Jacobian is formed by combining the unit actuator direction with the moment arm generated by platform rotation.

$$p_i = R_{zyx} P_i$$

$$r_i = p_i + t - B_i$$

$$l_i = \|r_i\|$$

$$u_i = \frac{r_i}{l_i}$$

$$J = \begin{bmatrix} u_1^T & (p_1 \times u_1)^T \\ u_2^T & (p_2 \times u_2)^T \\ \vdots & \vdots \\ u_6^T & (p_6 \times u_6)^T \end{bmatrix} \quad (5)$$

The resulting  $6 \times 6$  Jacobian matrix relates platform twist to actuator velocity and forms the basis for the differential kinematics used during motion propagation. The Jacobian becomes singular when the platform approaches certain configurations, resulting in reduced control authority or amplification of actuator errors. In future work, singular Jacobian detection will be integrated.

### 3. Differential Kinematics

Differential kinematics provide a means of relating velocities between the end effector pose and the individual actuator lengths of the Stewart Platform. While inverse kinematics determine the actuator lengths required to achieve a desired pose, differential kinematics describe how small changes in pose correspond to actuator velocities. This relationship is defined through the manipulator Jacobian (Equation 6).

$$\begin{aligned} \dot{l} &= J(X)V \\ V &= \begin{bmatrix} v \\ \omega \end{bmatrix} \end{aligned} \tag{6}$$

where:  $v$  : linear velocity  
 $\omega$  : angular velocity

Here  $\dot{l}$  represents the actuator velocity vector,  $J(X)$  is the Stewart platform Jacobian evaluated at pose  $X$ , and  $V$  is the platform twist containing both linear and angular velocity components.

In this work, differential kinematics are used to propagate the platform state during motion between two poses. Given the current state and a goal state, a desired platform twist is first computed based on the position and orientation error. Proportional gains are applied separately to translational and rotational components to produce a desired linear and angular velocity for the platform.

The Jacobian of the Stewart platform is then used to convert this desired end-effector twist into actuator velocities. Because the physical actuators have maximum velocity limits, the resulting actuator speeds are checked and uniformly scaled if necessary to ensure that no actuator exceeds its allowable velocity.

Once valid actuator velocities are obtained, the corresponding platform motion is recomputed using the pseudoinverse of the Jacobian. This corrected twist is integrated over the timestep to update the platform pose. In this way, the differential kinematics provide a consistent mapping between platform motion and actuator behavior while respecting the physical constraints of the mechanism.

---

#### Algorithm 7 Differential Kinematics

---

```

1: function DKSTEP( $x, x_{target}, dt$ )
2:    $dx \leftarrow x_{target} - x$ 
3:    $K_{p,pos} \leftarrow 1$ 
4:    $K_{p,rot} \leftarrow 0.5$ 
5:    $v \leftarrow K_{p,pos} * dx(1 : 3)$ 
6:    $\omega \leftarrow K_{p,rot} * dx(4 : 6)$ 
7:    $V \leftarrow \{v; \omega\}$ 
8:    $J \leftarrow \text{JACOBIAN}(x)$ 
9:    $ldot \leftarrow J * V$ 
10:   $maxSpeed \leftarrow \text{MAX}(\text{ABS}(ldot))$ 
11:  if  $maxSpeed > \text{LINEARACTUATOR.vel}_{max}$  then
12:     $scale \leftarrow \text{LINEARACTUATOR.vel}_{max} / maxSpeed$ 
13:     $V \leftarrow V * scale$ 
14:  end if
15:   $ldot \leftarrow J * V$ 
16:   $V_{corrected} = \text{INV}(J) * ldot$ 
17:   $x_{next} \leftarrow x + V_{corrected} * dt$ 
18:   $F \leftarrow I_6$ 
19:  return  $x_{next}, F$ 
20: end function

```

---

## VI. Belief Propagation

Belief propagation refers to the process of estimating the state of a system over time using uncertain measurements and a mathematical model of system dynamics. Instead of computing the exact state, we maintain a *belief*, which is a

probability distribution over the possible states.

As the system evolves, two main operations occur repeatedly: prediction and correction. During prediction, the current belief is propagated through the system's motion model. During correction, sensor measurements are incorporated to refine the estimate. The Extended Kalman Filter (EKF) is a widely used algorithm that performs this process for nonlinear systems by linearizing the dynamics around the current estimate.

### A. Process and Observation Noise

Real-world systems are affected by uncertainty in both their motion and sensing processes. These uncertainties are modeled as random noise.

Process noise represents uncertainty in the system dynamics. Even if we apply a control input, the resulting state may differ due to unmodeled effects, disturbances, or imperfect actuators. This is typically modeled as Gaussian noise with covariance matrix  $Q$ .

Observation noise represents uncertainty in sensor measurements. This is due to factors like environmental disturbances or imperfect sensors. This noise is also typically modeled as Gaussian with covariance matrix  $R$ .

The system can therefore be represented as:

$$x_k = f(x_{k-1}, u_k) + w_k$$

$$z_k = h(x_k) + v_k$$

where:

- $x_k$  is the system state
- $u_k$  is the control input
- $z_k$  is the measurement
- $w_k \sim \mathcal{N}(0, Q)$  is process noise
- $v_k \sim \mathcal{N}(0, R)$  is observation noise

These noise models determine how uncertainty grows during prediction and how strongly measurements influence the correction step.

### B. Error Covariance

The uncertainty in the state estimate is captured by the error covariance matrix, denoted  $P$ . This matrix describes how uncertain the filter is about each state variable and how errors in different variables are correlated.

During the prediction step, uncertainty increases because process noise is introduced. The covariance propagates as

$$P_k^- = F_k P_{k-1} F_k^T + Q_k$$

where:

- $F_k$  is the Jacobian of the motion model
- $Q_k$  is the process noise covariance

When a measurement is incorporated, the covariance is reduced because the observation provides new information. The updated covariance becomes

$$P_k = (I - K_k H_k) P_k^-$$

where:

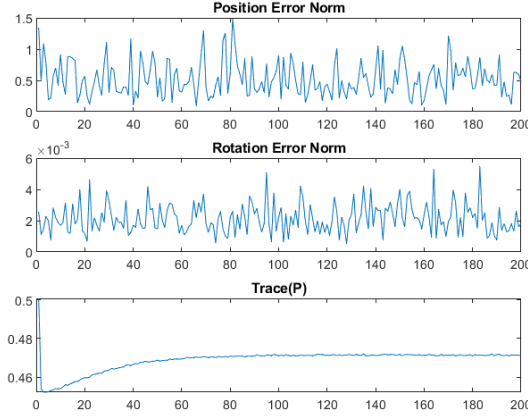
- $H_k$  is the Jacobian of the observation model
- $K_k$  is the Kalman gain

This matrix plays a critical role in balancing trust between the model prediction and incoming measurements.

### C. Extended Kalman Filter

The Extended Kalman Filter (EKF) is an extension of the Kalman Filter designed for nonlinear systems. Since many real-world models are nonlinear, the EKF approximates them by linearizing around the current state estimate using Jacobians.

The EKF operates in two main steps.



**Fig. 4** Testing of the implemented EKF using simulated measurement updates

### Prediction step

$$\hat{x}_k^- = f(\hat{x}_{k-1}, u_k)$$

$$P_k^- = F_k P_{k-1} F_k^T + Q_k$$

where  $F_k$  is the Jacobian of  $f$  evaluated at the current estimate.

### Update step

First compute the innovation:

$$y_k = z_k - h(\hat{x}_k^-)$$

Then compute the Kalman gain:

$$K_k = P_k^- H_k^T (H_k P_k^- H_k^T + R_k)^{-1}$$

Finally update the state and covariance:

$$\hat{x}_k = \hat{x}_k^- + K_k y_k$$

$$P_k = (I - K_k H_k) P_k^-$$

Through repeated prediction and update steps, the EKF continuously refines the estimate of the system state while accounting for both process and observation uncertainty.

To evaluate the performance of the Extended Kalman Filter (EKF), a simulation was constructed in which the true platform state, noisy measurements, and filter estimate were generated and compared over time. The goal of this experiment was to verify that the EKF can correctly estimate the pose of the Stewart platform despite measurement noise and an incorrect initial estimate. The simulation maintains two separate representations of the system: the *true state* of the platform and the *estimated state* maintained by the EKF. The true state evolves according to a simple motion model that drives the platform toward a desired goal pose. At each timestep, the platform pose is converted into actuator lengths using the inverse kinematics model of the Stewart platform. These actuator lengths represent the ideal sensor measurements. To simulate realistic conditions, Gaussian noise is added to the actuator lengths using the measurement covariance matrix  $R$ . The resulting noisy measurements are provided to the EKF as the sensor input.

The EKF then performs its standard prediction and update steps. The prediction step propagates the current state estimate toward the goal pose using the system motion model. The update step incorporates the noisy actuator measurements to correct the predicted state using the measurement model derived from the platform inverse kinematics. To test the filter's ability to converge, the initial estimated state is intentionally offset from the true state by a significant

|                |                                 |
|----------------|---------------------------------|
| $\Delta$       | 125 mm                          |
| $\gamma_{RRT}$ | 250 mm                          |
| Goal Bias      | 0.05                            |
| End Criteria   | time                            |
| Goal Tolerance | 1 mm                            |
| $c(x)$         | $g(parent) + \text{trace}(P_x)$ |
| $H(x)$         | 0                               |
| $Q$            | 2 mm, 2 rad                     |
| $R$            | 0.2904 mm                       |

**Table 2 Experimental Algorithm Parameters**

position and rotation error. Additionally, the initial covariance matrix  $P$  is set to a relatively large value to reflect high uncertainty in the starting estimate.

Two primary metrics are recorded during the simulation to evaluate the filter performance. The first metric is the estimation error between the true state and the estimated state. This error is computed separately for position and rotation. The position error is calculated as the Euclidean norm of the difference between the true and estimated position components, while the rotation error is calculated similarly using the rotational state components. These metrics indicate how accurately the filter tracks the platform pose over time. The second metric is the trace of the covariance matrix  $P$ . The trace of the covariance matrix provides a scalar measure of total state uncertainty. Monitoring the trace provides insight into how the filter's confidence in its estimate evolves during the simulation.

At the beginning of the simulation, the estimation error is relatively large due to the deliberately incorrect initial state estimate. As the EKF receives successive measurements, the filter uses the information contained in the actuator length observations to correct the predicted state. This results in a rapid reduction in both position and rotation error. Over time, the errors converge toward small values, demonstrating that the EKF is able to recover the correct platform pose despite starting from an inaccurate initial condition and receiving noisy measurements. This behavior confirms that the measurement model and Jacobians used by the filter are correctly implemented and that the sensor information is sufficient to constrain the platform state.

The trace of the covariance matrix also decreases during the early stages of the simulation. This indicates that the filter becomes increasingly confident in its estimate as more measurements are incorporated. After convergence, the covariance stabilizes at a lower value, reflecting the steady-state uncertainty determined by the process and measurement noise levels.

Overall, the results show that the Extended Kalman Filter successfully tracks the true platform state and maintains a bounded level of estimation uncertainty. This validates the EKF implementation for use in estimating the pose of the Stewart platform from noisy actuator measurements.

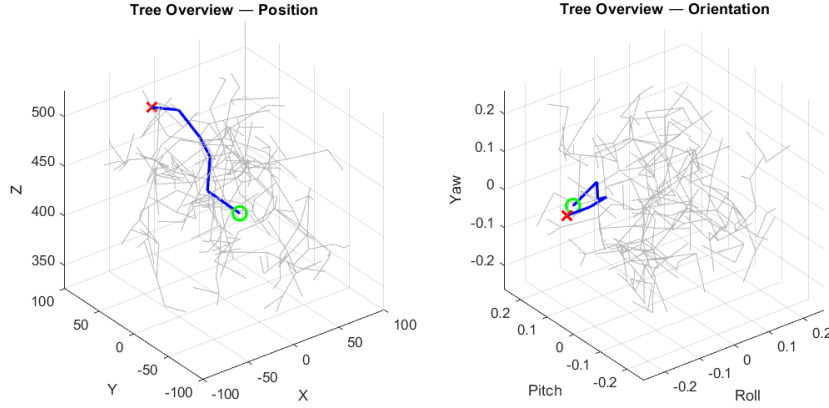
## VII. Experimental Setup

To evaluate the performance of the proposed planning framework, simulation experiments were conducted using the Stewart platform model described previously with realistic actuator constraints and sensor uncertainty. For each experiment, start and goal configurations were randomly generated within the valid workspace of the platform while respecting actuator constraints. Motion planning was then performed using the RRT, RRT\*, and RRT# algorithms producing trees and paths similar to those shown in Figure 5.

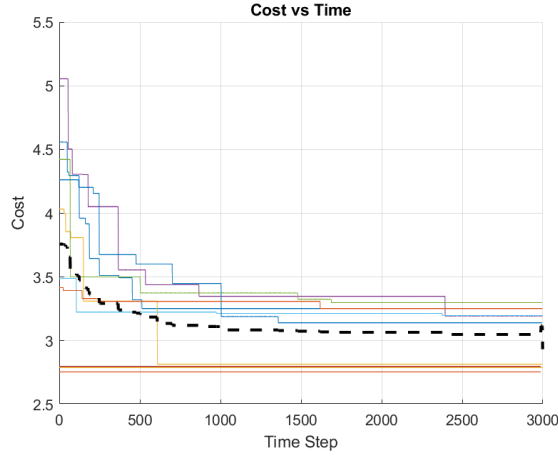
Each algorithm was executed for a fixed planning duration, during which the planner continuously expanded the search tree and updated the best available trajectory to the goal configuration. Multiple trials were conducted in order to account for the stochastic nature of sampling-based planning algorithms.

The primary objective of the experiments is to analyze the behavior of uncertainty along a Stewart Platform trajectory in order to determine if the trace of error covariance is an appropriate cost function and the degree to which it promotes uncertainty reduction along the trajectory when used in geometrically optimized SBM algorithms.

An additional goal of the experiments is to identify qualitatively if there is a relationship between path uncertainty and path distance. This would help to determine whether or not path length contributes to growth uncertainty or if they are independent.



**Fig. 5** Example trajectory generated by the planner in the Stewart platform configuration space.



**Fig. 6** Evolution of path uncertainty over time across ten trials using RRT#.

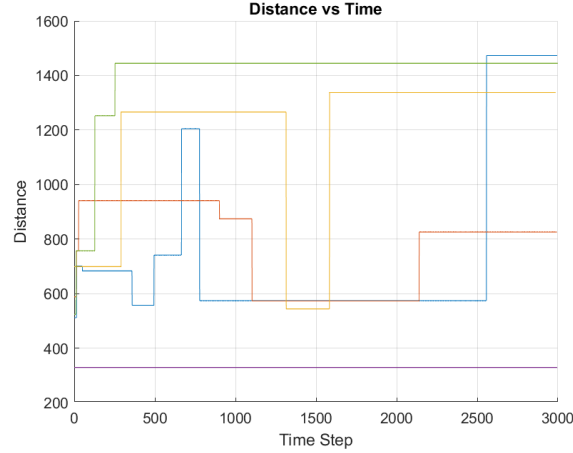
## VIII. Discussion of Results

Preliminary experiments were conducted with five trials of five minutes each to qualitatively evaluate convergence behavior. As shown in Figure 6, the planner consistently identifies trajectories with decreasing uncertainty over time, demonstrating the viability of using the trace of the error covariance matrix as a cost function.

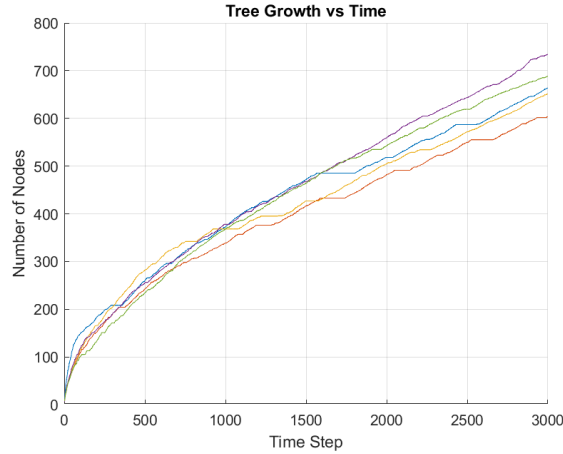
The convergence behavior closely resembles that observed when using path distance as the cost metric, as shown in Figure 1. However, the rate of convergence with respect to time is noticeably slower when uncertainty is used as the cost function. This is expected due to the additional computational complexity associated with propagating covariance through the Extended Kalman Filter during path evaluation.

Ideally, the uncertainty-cost trials would also be normalized to allow direct comparison with the distance-based cost function. However, unlike path distance, where the optimal solution is known to be the straight-line trajectory between start and goal states, the least-uncertain path cannot be determined analytically. In principle, the trials could be normalized with respect to the lowest-uncertainty trajectory discovered during experimentation. However, due to the stochastic nature of sampling-based motion planning algorithms, it cannot be definitively proven that the best path found during experimentation is the globally optimal solution.

One of the more interesting results produced during experimentation is the behavior of path distance when uncertainty is used as the optimization metric. As shown in Figure 7, the evolution of path length exhibits no clear correlation with path uncertainty. Path length both increases and decreases over time across trials, indicating that minimizing uncertainty



**Fig. 7** Evolution of path length over time when trace of uncertainty covariance is used as the cost function.



**Fig. 8** Growth of the planning tree over time when using RRT#.

does not necessarily correspond to minimizing path distance.

Another important factor affecting convergence speed is the integration of rejection sampling and heuristic pruning conditions. When path distance is used as the cost function, rejection sampling and heuristic constraints prevent sampling and rewiring through nodes that cannot produce shorter paths. This significantly improves convergence speed by avoiding unnecessary computation during tree expansion and rewiring.

However, because the experiments show no clear correlation between path length and path uncertainty, applying traditional distance-based rejection sampling and heuristics was deemed inappropriate for this investigation. Further research could explore alternative heuristic definitions tailored specifically for uncertainty-based cost functions.

Finally, the tree growth curves shown in Figure 8 exhibit behavior typical of sampling-based motion planning algorithms. Initially, the number of nodes increases rapidly as the tree expands through unexplored regions of the configuration space. Over time, the growth rate gradually decreases as the tree becomes denser and nearest-neighbor searches and successor rewiring operations become computationally more expensive.

## IX. Conclusion

This work investigated uncertainty-aware motion planning for Stewart platforms operating in a six-degree-of-freedom configuration space. Traditional sampling-based motion planners typically optimize geometric objectives, primarily path length, which does not directly account for the estimation uncertainty that accumulates during robot motion.

For high-precision robotic systems, particularly those used in aerospace applications such as in-orbit assembly and manipulation, maintaining accurate state knowledge throughout motion is critical.

To address this challenge, a motion planning framework was developed that incorporates belief propagation through an Extended Kalman Filter (EKF). The EKF propagates the covariance of the platform state along candidate trajectories, allowing the trace of the error covariance matrix to be used as a cost metric for trajectory evaluation. In this formulation, paths are evaluated not only by their geometric properties but also by the uncertainty associated with executing the motion.

Sampling-based planning algorithms from the Rapidly-Exploring Random Tree (RRT) family were evaluated to determine their suitability for this task. Comparative experiments showed that RRT# consistently achieved superior convergence behavior relative to RRT and RRT\*, making it the most effective planner for this investigation. When uncertainty propagation was incorporated into the planning process, the planner was able to identify trajectories with progressively lower estimation uncertainty over time.

Experimental results also indicate that path length and path uncertainty are not strongly correlated. When uncertainty was used as the optimization metric, path distance both increased and decreased during optimization, suggesting that minimizing trajectory uncertainty may require paths that differ significantly from those obtained through purely geometric optimization.

Overall, the results demonstrate that incorporating belief propagation into sampling-based motion planning provides a practical method for generating trajectories that prioritize estimation confidence rather than geometric efficiency. This approach offers a promising direction for improving the reliability of precision robotic manipulation systems where state uncertainty plays a critical role in task success.

## **X. Future Work**

This investigation highlighted several opportunities to further improve and extend the planning framework presented.

The current implementation used a simplified linearized state transition model to propagate uncertainty through the Extended Kalman Filter. Future work would incorporate a more accurate linearization of the system dynamics by explicitly computing the Jacobian of the differential kinematics model. This would improve the fidelity of the covariance propagation and provide a more accurate estimate of trajectory uncertainty.

Another important extension could include the incorporation of Jacobian singularity detection and avoidance. Stewart platforms experience configurations in which the manipulator Jacobian becomes singular or poorly conditioned, leading to loss of controllability or amplification of actuator errors. Integrating singularity detection into the planner would allow such configurations to be avoided during trajectory generation.

The computational cost of uncertainty propagation also limits the speed of the planning process. In this work, traditional rejection sampling and heuristic pruning strategies were not applied because path distance was found to have little correlation with trajectory uncertainty. Future research could explore the development of heuristic functions specifically designed for uncertainty-based cost metrics, to enable a more efficient exploration of the configuration space.

Additionally, the experiments conducted in this work were limited to simulation. Future validation on physical hardware would provide insight into how modeling assumptions, actuator dynamics, and sensor characteristics influence uncertainty propagation in practice.

Finally, the relationship between geometric path characteristics and estimation uncertainty remains an open question. Further experimentation with larger datasets and alternative platform geometries may help reveal patterns in how platform motion influences the growth of state uncertainty. Understanding this relationship could lead to the development of improved planning strategies tailored specifically for precision robotic systems.

## **Acknowledgments**

The author would like to acknowledge Ryan Erandis for creating the foundation of uncertainty path planning for Stewart Platforms, which this investigation built on. The author also thanks Dr. Michael Otte for his guidance and support throughout this work. It must also be acknowledged that artificial intelligence was used in the construction of this document for the purposes of formatting and proof reading to insure compliance with AIAA guidelines.

## References

- [1] Ernandis, R., “Sampling Based Motion Planning for Minimizing Position Uncertainty with Stewart Platforms,” Master’s thesis, University of Maryland, 2020.
- [2] Merlet, J. P., *Parallel Robots*, 2<sup>nd</sup> ed., Springer, 2006.
- [3] LaValle, S. M., “Rapidly-Exploring Random Trees: A New Tool for Path Planning,” Tech. rep., Department of Computer Science, Iowa State University, 1998.
- [4] Karaman S, F. E., “Sampling-based Algorithms for Optimal Motion Planning,” *International Journal of Robotics Research*, 2011.
- [5] Arslan O, P., Tsiotras, “Use of Relaxation Methods in Sampling-Based Algorithms for Optimal Motion Planning,” *IEEE International Conference on Robotics and Automation*, 2013.